

苏州大学实验报告

院系	计算机学院	年级专业	21 计科	姓名	方浩楠	学号	2127405048
课程名称	操作系统课程实践					成绩	
指导教师	王红玲	同组实验者	无	实验日期	2023. 4. 17		

实验名称 实验 7 虚拟内存实验

一. 实验目的

- (1) 理解操作系统中缺页中断的工作原理。
- (2) 学会通过修改内核实现统计系统缺页次数的方法。
- (3) 学会观察/proc 中有关虚拟内存的内容。
- (4) 学会通过使用相关工具来统计一段时间内的缺页次数

二. 实验内容

- (1) 通过修改 Linux 内核中相关代码, 统计系统缺页次数。
- (2) 通过查看/proc/vmstat 的变化来统计一段时间的缺页次数

三. 实验步骤和结果

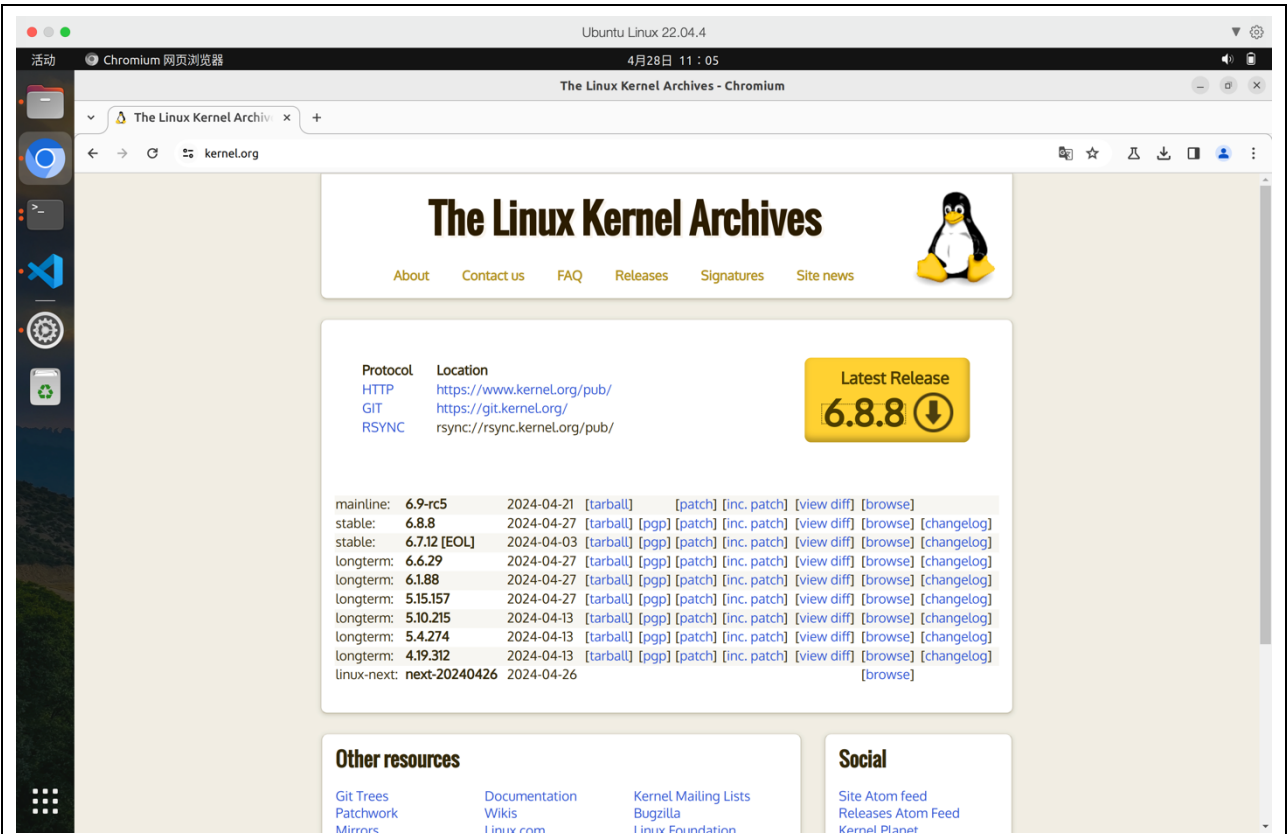
1. 通过修改 Linux 内核中相关代码, 统计系统缺页次数

系统环境:

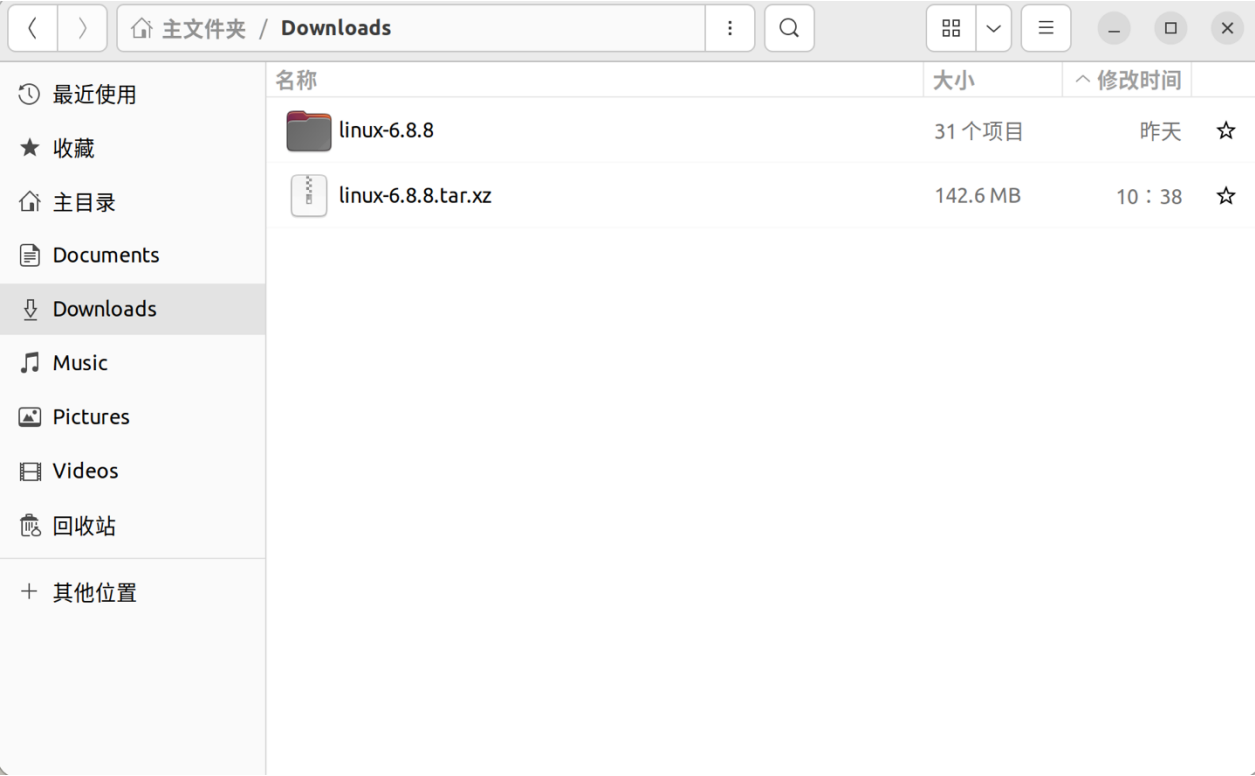
```
fhn@ubuntu: /usr/src/linux-headers-5.15.0-101-generic
fhn@ubuntu: /usr/src/linux-headers-5.15.0-101-generic$ neofetch
      .-/+00ssss00+/-
      ':+ssssssssssssssssss+:
      +ssssssssssssssssssyyssss+-
      .ossssssssssssssssssdMMMNysssso.
      /ssssssssssssshdnnNNmyNNMMhssssss/
      +ssssssssshmydMMNNNNNNdddyssssss++
      /ssssssssshNMMMyhhyyyyhNMMMNhssssss/
      .ssssssssdMMNNhssssssshNMMMdssssss+.
      +ssssshhhyNMMNyssssssssssyNMMMyssssss+
      ossyNMMNNyMMhssssssssssshmmhssssssso
      ossyNMMNNyMMhssssssssssshmmhssssssso
      +ssssshhhyNMMNyssssssssssyNMMMyssssss+
      .ssssssssdMMNNhssssssshNMMMdssssss+.
      /ssssssssshNMMMyhhyyyyhNMMMNhssssss/
      +ssssssssdnydMMNNNNNNdddyssssss++
      /ssssssssshdnnNNmyNNMMhssssss/
      .ossssssssssssssssssdMMMNysssso.
      +ssssssssssssssssssyyssss+-
      ':+ssssssssssssssssss+:
      .-/+00ssss00+/-
      _-_-_-_-_-_-_-_-_-_-_-_-_-_-_-_-
      OS: Ubuntu 22.04.4 LTS aarch64
      Host: Parallels ARM Virtual Machine 0.1
      Kernel: 5.15.0-101-generic
      Uptime: 13 mins
      Packages: 1835 (dpkg), 12 (snap)
      Shell: bash 5.1.16
      Resolution: 3024x1890
      DE: GNOME 42.9
      WM: Mutter
      WM Theme: Adwaita
      Theme: Yaru [GTK2/3]
      Icons: Yaru [GTK2/3]
      Terminal: gnome-terminal
      CPU: (2)
      GPU: 00:0a.0 Red Hat, Inc. Virtio GPU
      Memory: 1386MiB / 1967MiB
      _-_-_-_-_-_-_-_-_-_-_-_-_-_-_-_-
      _-_-_-_-_-_-_-_-_-_-_-_-_-_-_-_-
```

Linux 系统中安装了 vscode.使用了 vscode 而不是 vim 来打开文本文件

下载 Linux Kernel:



下载完成后解压:



打开内核文件夹,使用 vscode 打开 include/linux/mm.h

```
1. code ./include/linux/mm.h
```

添加:

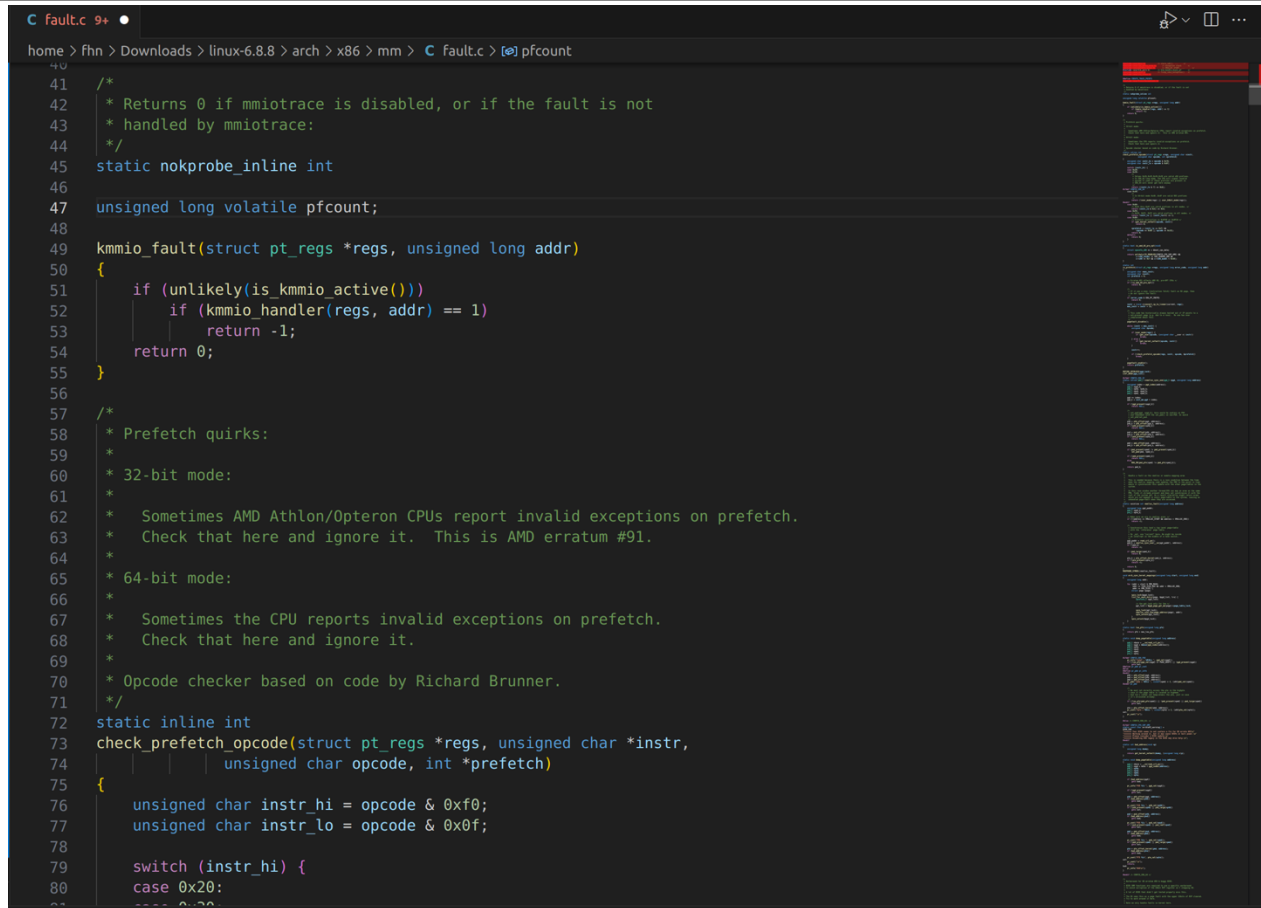
1. `extern unsigned long volatile pfcount;`

然后使用 vscode 打开 `arch/x86/mm/fault.c`

1. code `./arch/x86/mm/fault.c`

在文件中定义变量:

1. `unsigned long volatile pfcount;`



```
C fault.c 9+
home > fhn > Downloads > linux-6.8.8 > arch > x86 > mm > C fault.c > [e] pfcount
41  /*
42  * Returns 0 if mmiotrace is disabled, or if the fault is not
43  * handled by mmiotrace:
44  */
45  static nokprobe_inline int
46
47  unsigned long volatile pfcount;
48
49  kmmio_fault(struct pt_regs *regs, unsigned long addr)
50  {
51      if (unlikely(!is_kmmio_active()))
52          if (kmmio_handler(regs, addr) == 1)
53              return -1;
54      return 0;
55  }
56
57  /*
58  * Prefetch quirks:
59  *
60  * 32-bit mode:
61  *
62  * Sometimes AMD Athlon/Opteron CPUs report invalid exceptions on prefetch.
63  * Check that here and ignore it. This is AMD erratum #91.
64  *
65  * 64-bit mode:
66  *
67  * Sometimes the CPU reports invalid exceptions on prefetch.
68  * Check that here and ignore it.
69  *
70  * Opcode checker based on code by Richard Brunner.
71  */
72  static inline int
73  check_prefetch_opcode(struct pt_regs *regs, unsigned char *instr,
74                      unsigned char opcode, int *prefetch)
75  {
76      unsigned char instr_hi = opcode & 0xf0;
77      unsigned char instr_lo = opcode & 0x0f;
78
79      switch (instr_hi) {
80      case 0x20:
81          if (instr_lo & 0x04)
82              *prefetch = 1;
83          return 0;
84      case 0x21:
85          if (instr_lo & 0x04)
86              *prefetch = 1;
87          return 0;
88      case 0x22:
89          if (instr_lo & 0x04)
90              *prefetch = 1;
91          return 0;
92      case 0x23:
93          if (instr_lo & 0x04)
94              *prefetch = 1;
95          return 0;
96      case 0x24:
97          if (instr_lo & 0x04)
98              *prefetch = 1;
99          return 0;
100     case 0x25:
101         if (instr_lo & 0x04)
102             *prefetch = 1;
103         return 0;
104     case 0x26:
105         if (instr_lo & 0x04)
106             *prefetch = 1;
107         return 0;
108     case 0x27:
109         if (instr_lo & 0x04)
110             *prefetch = 1;
111         return 0;
112     case 0x28:
113         if (instr_lo & 0x04)
114             *prefetch = 1;
115         return 0;
116     case 0x29:
117         if (instr_lo & 0x04)
118             *prefetch = 1;
119         return 0;
120     case 0x2a:
121         if (instr_lo & 0x04)
122             *prefetch = 1;
123         return 0;
124     case 0x2b:
125         if (instr_lo & 0x04)
126             *prefetch = 1;
127         return 0;
128     case 0x2c:
129         if (instr_lo & 0x04)
130             *prefetch = 1;
131         return 0;
132     case 0x2d:
133         if (instr_lo & 0x04)
134             *prefetch = 1;
135         return 0;
136     case 0x2e:
137         if (instr_lo & 0x04)
138             *prefetch = 1;
139         return 0;
140     case 0x2f:
141         if (instr_lo & 0x04)
142             *prefetch = 1;
143         return 0;
144     }
145     return 1;
146 }
```

并在 `do_page_fault()`函数中找到并修改 `good_area`

```
1. good_area:
2.     if (unlikely(access_error(error_code, vma))) {
3.         bad_area_access_error(regs, error_code, address, vma); return;
4.     }
5. pfcount++;
6.
```

```

1290
1291
1292 good_area:
1293     if (unlikely(access_error(error_code, vma))) {
1294         bad_area_access_error(regs, error_code, address, vma); return;
1295     }
1296     pfcnt++;
1297
1298

```

在 kernel/kallsyms.c 文件最后插入:

1. EXPORT_SYMBOL(pfcnt);

```

home > /bin > Downloads > linux-6.8.8 > kernel > C kallsyms.c
964 static int kallsyms_open(struct inode *inode, struct file *file)
965 {
966     return 0;
967 }
968
969 #ifdef CONFIG_KGDB_KDB
970 const char *kdb_walk_kallsyms(loff_t *pos)
971 {
972     static struct kallsym_iter kdb_walk_kallsyms_iter;
973     if (*pos == 0) {
974         memset(&kdb_walk_kallsyms_iter, 0,
975             sizeof(kdb_walk_kallsyms_iter));
976         reset_iter(&kdb_walk_kallsyms_iter, 0);
977     }
978     while (1) {
979         if (!update_iter(&kdb_walk_kallsyms_iter, *pos))
980             return NULL;
981         ++*pos;
982         /* Some debugging symbols have no name. Ignore them. */
983         if (kdb_walk_kallsyms_iter.name[0])
984             return kdb_walk_kallsyms_iter.name;
985     }
986 }
987 #endif /* CONFIG_KGDB_KDB */
988
989 static const struct proc_ops kallsyms_proc_ops = {
990     .proc_open = kallsyms_open,
991     .proc_read = seq_read,
992     .proc_lseek = seq_lseek,
993     .proc_release = seq_release_private,
994 };
995
996 static int __init kallsyms_init(void)
997 {
998     proc_create("kallsyms", 0444, NULL, &kallsyms_proc_ops);
999     return 0;
1000 }
1001
1002 device_initcall(kallsyms_init);
1003 EXPORT_SYMBOL(pfcnt);

```

然后重新编译内核:

1. \$ make mrproper
2. \$ make clean
3. \$ make menuconfig
4. \$ make modules -j8
5. \$ make modules_in

2. 通过查看/proc/vmstat 的变化来统计一段时间的缺页次数
编写 page_fault_counter.py
其中统计了 10s 内的缺页次数.在终端打印进度条来增强体验

1. #!/usr/bin/env python3
2. import sys
3. import time
4. import re

```

5.
6. def get_page_fault():
7.     """获取当前系统的缺页中断次数。"""
8.     try:
9.         with open('/proc/vmstat', 'r') as vmstat:
10.             content = vmstat.read()
11.             match = re.search(r'pgfault\s+(\d+)', content)
12.             if match:
13.                 return int(match.group(1))
14.     except Exception as e:
15.         print(f"Error reading vmstat: {e}")
16.         return -1
17.
18. def print_progress_bar(duration):
19.     """显示一个简单的进度条，持续给定的秒数。"""
20.     for i in range(duration + 1):
21.         percent = (i / duration) * 100
22.         bar = '#' * int(percent // 2) + '-' * (50 - int(percent // 2))
23.         sys.stdout.write(f'\r[{bar}] {i}/{duration} seconds ({percent:.0f}%)')
24.         sys.stdout.flush()
25.         time.sleep(1)
26.     print() # Move to the next line
27.
28. def main():
29.     calc_time = 10 # 默认休眠时间
30.     if len(sys.argv) > 1:
31.         try:
32.             calc_time = int(sys.argv[1])
33.         except ValueError:
34.             print("Invalid input. Using default duration of 10 seconds.")
35.
36.     initial_faults = get_page_fault()
37.     if initial_faults < 0:
38.         print('Error reading initial page faults!')
39.         sys.exit(1)
40.
41.     print("Starting to monitor page faults...")
42.     print_progress_bar(calc_time)
43.
44.     final_faults = get_page_fault()
45.     if final_faults < 0:
46.         print('Error reading final page faults!')
47.         sys.exit(1)
48.

```

```

49.     faults_diff = final_faults - initial_faults
50.     print(f"Initial page faults: {initial_faults}")
51.     print(f"Final page faults: {final_faults}")
52.     print(f"Page faults occurred during the interval: {faults_diff}")
53.
54. if __name__ == '__main__':
55.     main()
56.

```

```

page_fault_counter.py
1  #!/usr/bin/env python3
2  import sys
3  import time
4  import re
5
6  def get_page_fault():
7      """获取当前系统的缺页中断次数。"""
8      try:
9          with open('/proc/vmstat', 'r') as vmstat:
10             content = vmstat.read()
11             match = re.search(r'pgfault\s+(\d+)', content)
12             if match:
13                 return int(match.group(1))
14     except Exception as e:
15         print(f"Error reading vmstat: {e}")
16         return -1
17
18 def print_progress_bar(duration):
19     """显示一个简单的进度条，持续给定的秒数。"""
20     for i in range(duration + 1):
21         percent = (i / duration) * 100
22         bar = '#' * int(percent // 2) + '.' * (50 - int(percent // 2))
23         sys.stdout.write(f'\r[{bar}] {i}/{duration} seconds ({percent:.0f}%)')
24         sys.stdout.flush()
25         time.sleep(1)
26     print() # Move to the next line
27
28 def main():
29     calc_time = 10 # 默认休眠时间
30     if len(sys.argv) > 1:
31         try:
32             calc_time = int(sys.argv[1])
33         except ValueError:
34             print("Invalid input. Using default duration of 10 seconds.")
35
36     initial_faults = get_page_fault()
37     if initial_faults < 0:
38         print("Error reading initial page faults!")
39         sys.exit(1)
40
41     print("Starting to monitor page faults...")

```

代码运行结果:

```

fhn@ubuntuserver:~/OS_experiment/experiment7$ python3 page_fault_counter.py
Starting to monitor page faults...
[#####] 10/10 seconds (100%)
Initial page faults: 18831991
Final page faults: 18847294
Page faults occurred during the interval: 15303

```

发现系统缺页次数被成功统计

四. 实验总结

1. 通过本次实验,我理解了操作系统中缺页中断的工作原理,并尝试修改内核实现了统计系统缺页次数,并且重新编译内核.
2. 通过一个 unsigned long volatile pfcunt;来统计系统的缺页次数,统计的原理是在缺页是将该变量的值+1,最后可以输出该变量的值来判断系统的缺页次数
3. 选择使用 python 脚本: Python 的简洁语法使得编程初学者和专业开发者都能快速理解和使用.对于处理文件和字符串操作,Python 提供了直接且易于使用的方法,这对于本脚本中的操作特

别有用。并且 Python 自带了丰富的库，可以轻松地进行文件操作、正则表达式匹配和命令行参数解析等任务，无需依赖外部库。

4. 该 python 脚本通过读取 `/proc/vmstat` 文件，并使用正则表达式匹配 `pgfault` 的值，脚本能获取到当前的缺页中断次数。这是监控系统性能的一种方式，特别是在内存密集型任务中。用户可以指定脚本监测的时间长度（默认为 10 秒）。脚本在这段时间内暂停，并在结束时再次读取缺页次数。脚本在等待期间显示一个简单的文本进度条，让用户知道当前时间的进展。这提高了用户界面的友好性，尤其是在较长的等待时间。脚本计算开始和结束时的缺页中断数之差，输出这段时间内发生的总缺页次数。这对于系统管理员和开发者来说是有用的信息，可以帮助他们理解在特定工作负载下系统的表现。
5. 缺页次数统计是通过监测 Linux 系统中的 `/proc/vmstat` 文件实现的。`/proc/vmstat` 包含了许多与虚拟内存及页面交换相关的统计信息，其中 `pgfault` 项记录了自系统启动以来发生的总缺页中断数。缺页中断发生时，意味着程序访问的数据不在物理内存中，需要从磁盘等更慢的存储介质加载到内存中。该方法的合理性在于直接使用了内核提供的数据，这些数据实时更新且准确反映了系统的内存管理状态。
6. 验证实验结果的准确性：多次运行脚本，检查结果的一致性。缺页次数应该随系统活动变化而变化，反复的结果应显示出这一点。
7. 在 `/proc/` 下增加一个项的基本过程

在 Linux 中，`/proc` 文件系统是一个虚拟文件系统，它提供了一个窗口来查看内核内部的信息，同时也可以用来与内核进行交互。增加一个新的项到 `/proc` 涉及以下步骤：1. 编写内核模块：创建一个新的内核模块，或修改现有模块。2. 实现文件操作函数：定义如 `open`, `read`, `write` 等操作的函数，这些函数决定了用户空间如何与该 `/proc` 文件交互。3. 注册 `proc` 文件：在模块的初始化函数中，使用 `proc_create` 函数创建一个新的 `proc` 条目。指定文件名和前面定义的文件操作函数。4. 清理工作：在模块的退出函数中，使用 `remove_proc_entry` 来删除 `proc` 文件条目。